并行算法

主讲人：段贵多
Email:duanguiduo@163.com

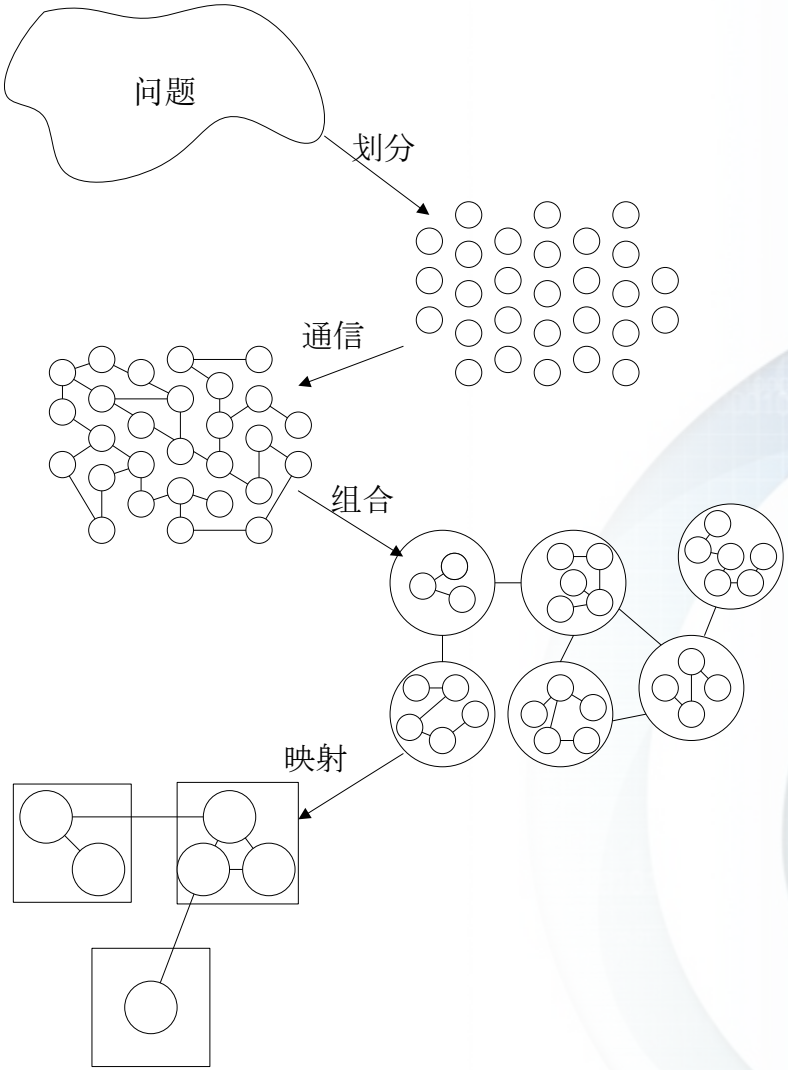
并行算法一般设计过程

1. PCAM设计过程
2. 划分
3. 通讯
4. 组合
5. 映射
6. 小结

PCAM设计过程

- 设计并行算法的四个阶段（PCAM方法学）
  - 划分(Partitioning)
  - 通讯(Communication)
  - 组合(Agglomeration)
  - 映射(Mapping)
- 划分：分解成小的任务，开拓并发性；
- 通讯：确定诸任务间的数据交换，监测划分的合理性；
- 组合：依据任务的局部性，组合成更大的任务；
- 映射：将每个任务分配到处理器上，提高算法的性能。

PCAM设计过程



1.划分

1.1 方法描述

1.2 域分解

1.3 功能分解

1.4 划分判据

划分——方法描述

- 充分开拓算法的并发性和可扩放性；
- 先进行数据分解(称域分解)，再进行计算功能的分解(称功能分解)；
- 使数据集和计算集互不相交；
- 划分阶段忽略处理器数目和目标机器的体系结构；
- 能分为两类划分：
  - 域分解(domain decomposition)
  - 功能分解(functional decomposition)

划分——域分解

- 划分的对象是数据，可以是算法的输入数据、中间处理数据和输出数据；
- 将数据分解成大致相等的小数据片；
- 划分时考虑数据上的相应操作；
- 如果一个任务需要别的任务中的数据，则会产生任务间的通讯；

域分解

❖示例：三维网格的域分解，各格点上计算都是重复的。下图是三种分解方法：

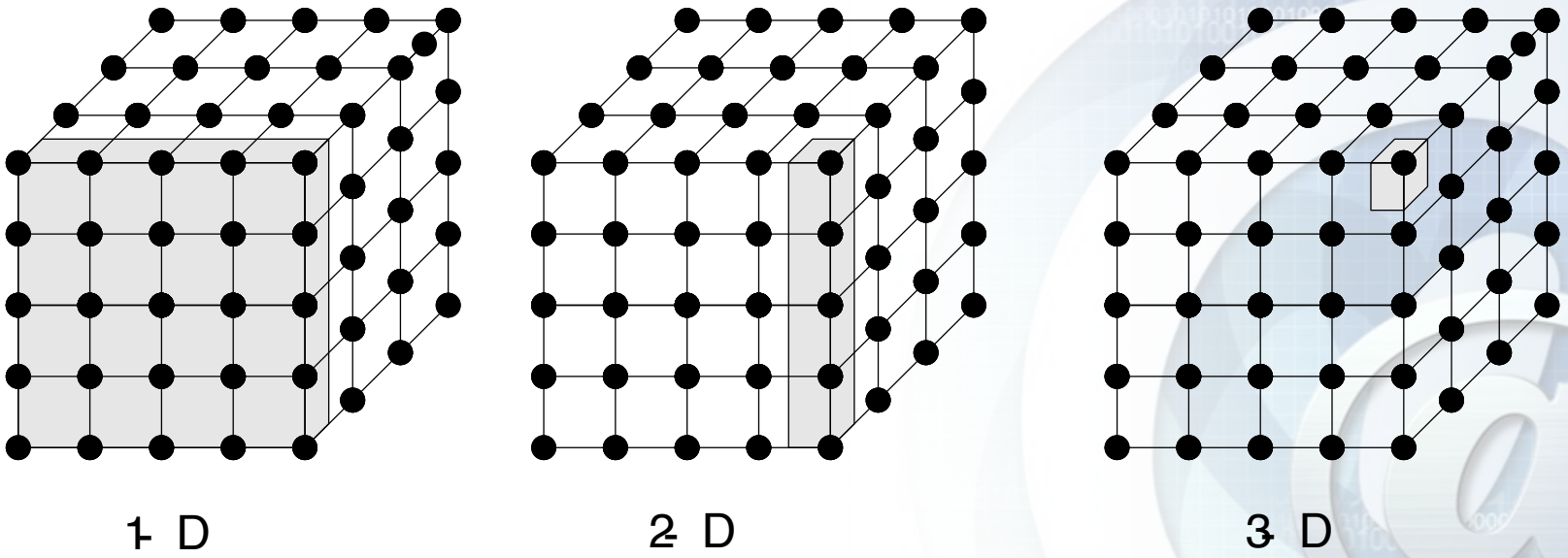


图7.2

域分解的经验方法

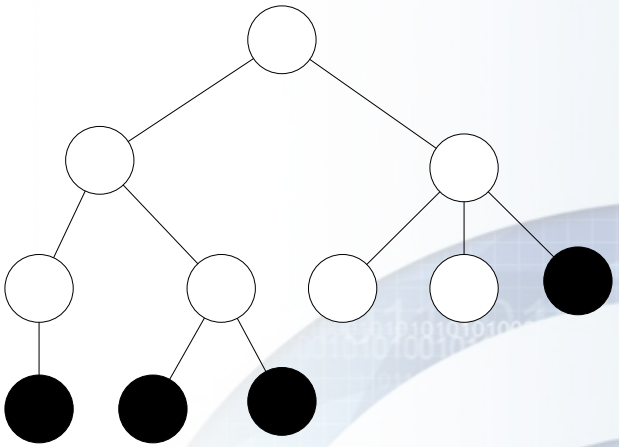
- 域分解的经验方法
- 优先集中在最大数据的划分；或者那些经常被访问的数据结构上；

功能分解

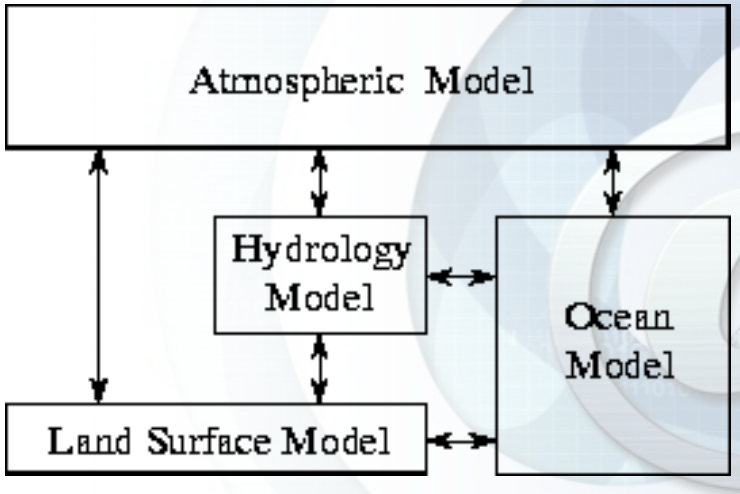
- 划分的对象是计算，将计算划分为不同的任务，其出发点不同于域分解；
- 划分后，研究不同任务所需的数据。如果这些数据不相交的，则划分是成功的；如果数据有相当的重叠，意味着要重新进行域分解和功能分解；
- 功能分解是一种更深层次的分解。

功能分解

❖ 示例1：搜索树



❖ 示例2：气候模型



划分判据

- 划分是否具有灵活性？
- 划分是否避免了冗余计算和存储？
- 划分任务尺寸是否大致相当？
- 任务数与问题尺寸是否成比例？
- 功能分解是一种更深层次的分解，是否合理？

2.通信

1.1 方法描述

1.2 四种通信模式

1.3 通信判据

通讯——方法描述

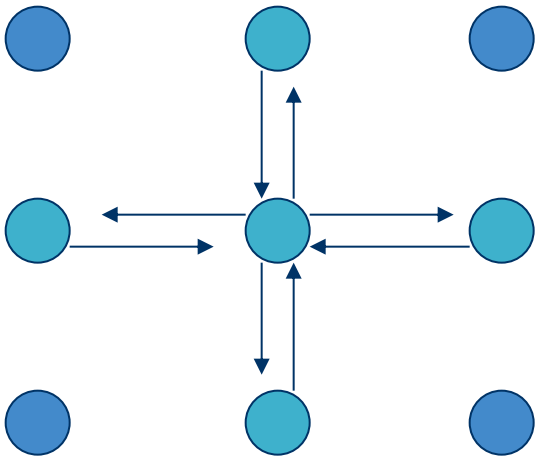
- 通讯是PCAM设计过程的重要阶段；
- 划分产生的诸任务，一般不能完全独立执行，需要在任务间进行数据交流；从而产生了通讯；
- 功能分解确定了诸任务之间的数据流；
- 诸任务是并发执行的，通讯则限制了这种并发性；

四种通讯模式

- 局部/全局通讯：局部通信时，每个任务只与较少的几个近邻居通信；全局通信中，每个任务与很多别的任务通信。
- 结构化/非结构化通讯：结构化通信时，一个任务和其近邻形成规整结构（如树、网格等）；非结构化通信中，通信网可能是任意图。
- 静态/动态通讯：通信伙伴的身份是否变化。
- 同步/异步通讯：接收方和发送方是否协同操作。

局部通讯

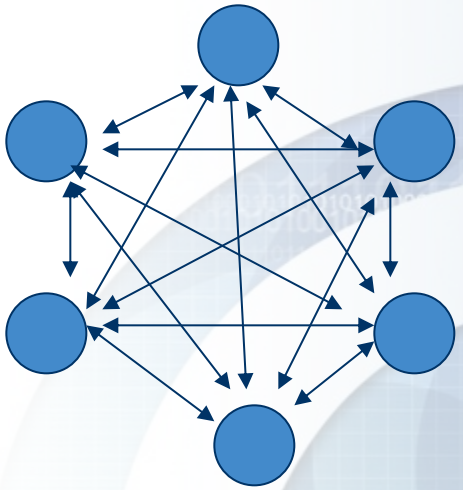
❖通讯限制在一个邻域内

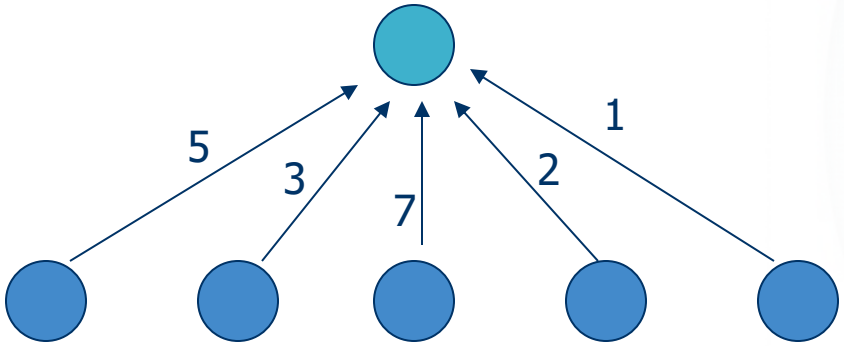


$$x_{i,j}(k)=\frac{4x_{i,j}(k-1)+x_{i-1,j}(k-1)+x_{i+1,j}(k-1)+x_{i,j-1}(k-1)+x_{i,j+1}(k-1)}{8}$$

全局通讯

- 通讯非局部的
- 例如：
  - All to All
  - Master-Worker



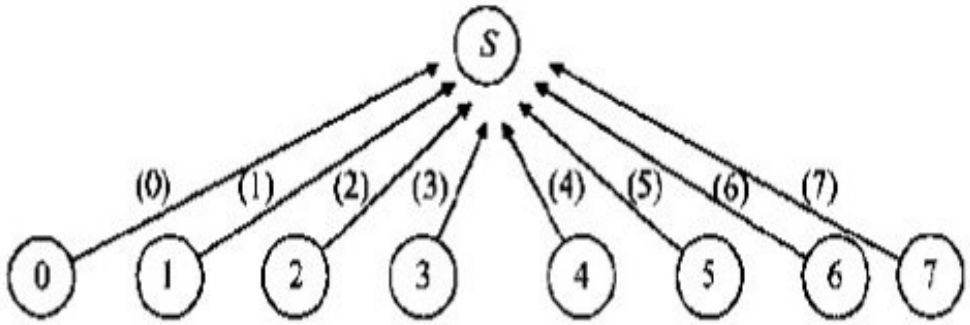


全局通信

❖一个顺序求和算法就呈现出全局通信的要求

一个顺序求和算法 $S = \sum_{i=0}^{N-1} x_i$，就呈现出全局通信要求。如图　　所示，一个根进程 S，负责从 n 个分布式任务中一次接收一个值并相加之：

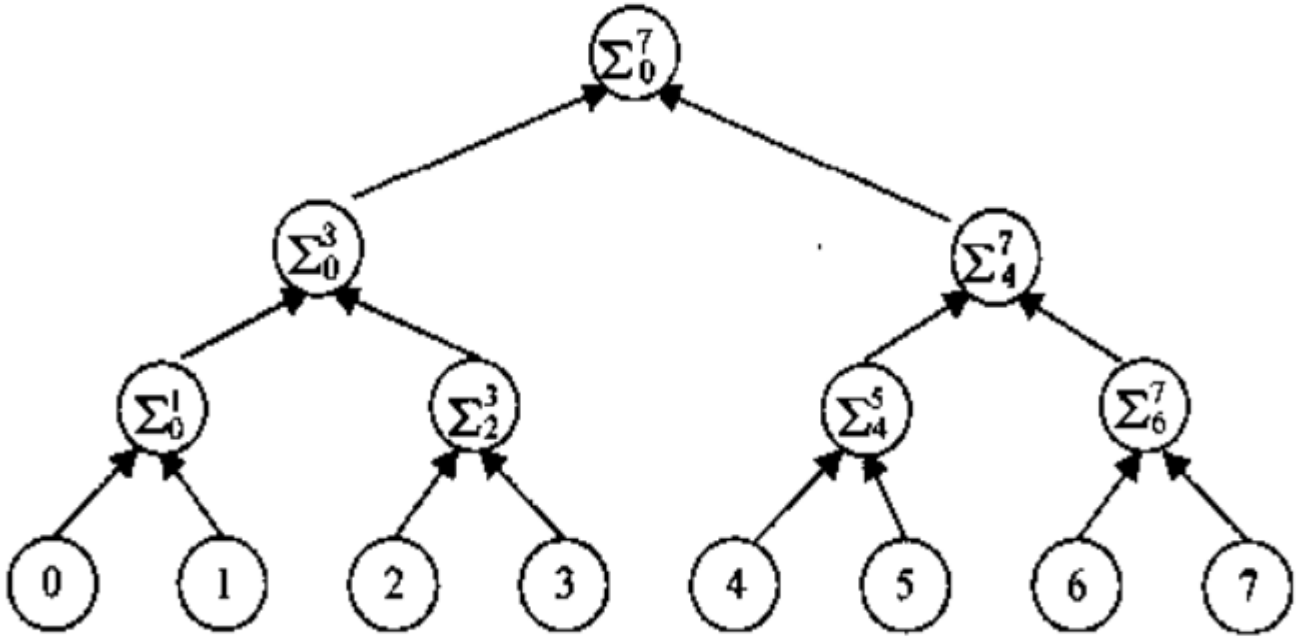
$$S_i = x_i + S_{i-1}$$



全局通信

❖一个顺序求和算法就呈现出全局通信的要求

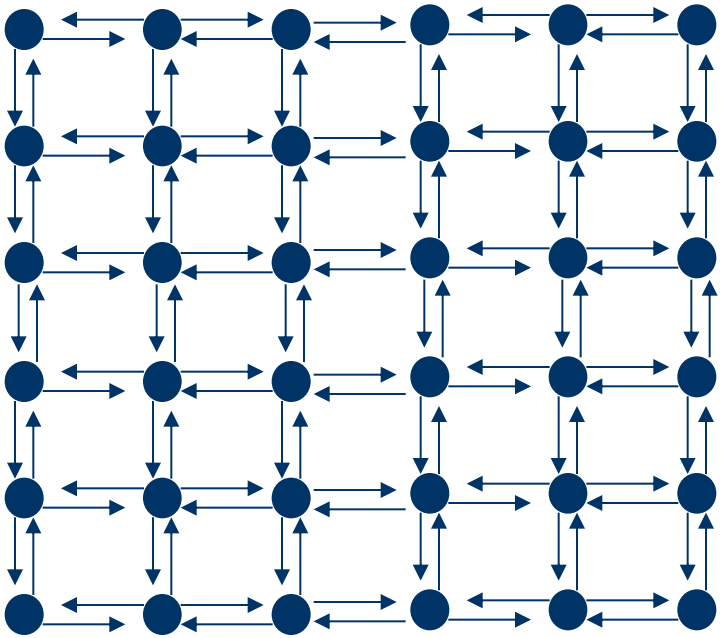
$$\sum_{i=0}^{2^n-1} = \sum_{i=0}^{2^{n-1}-1} + \sum_{i=2^{n-1}}^{2^n-1}$$

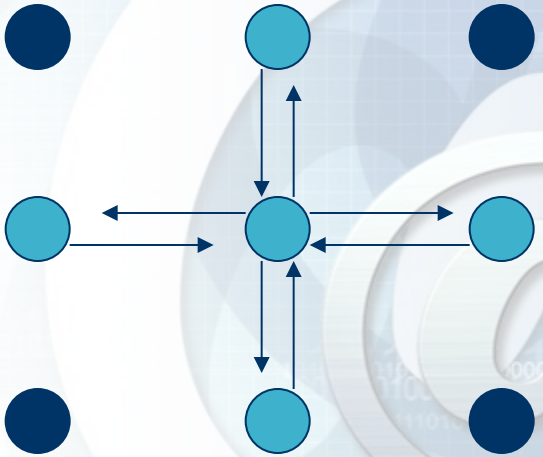


N=8的时候的分治求和树

结构化通讯

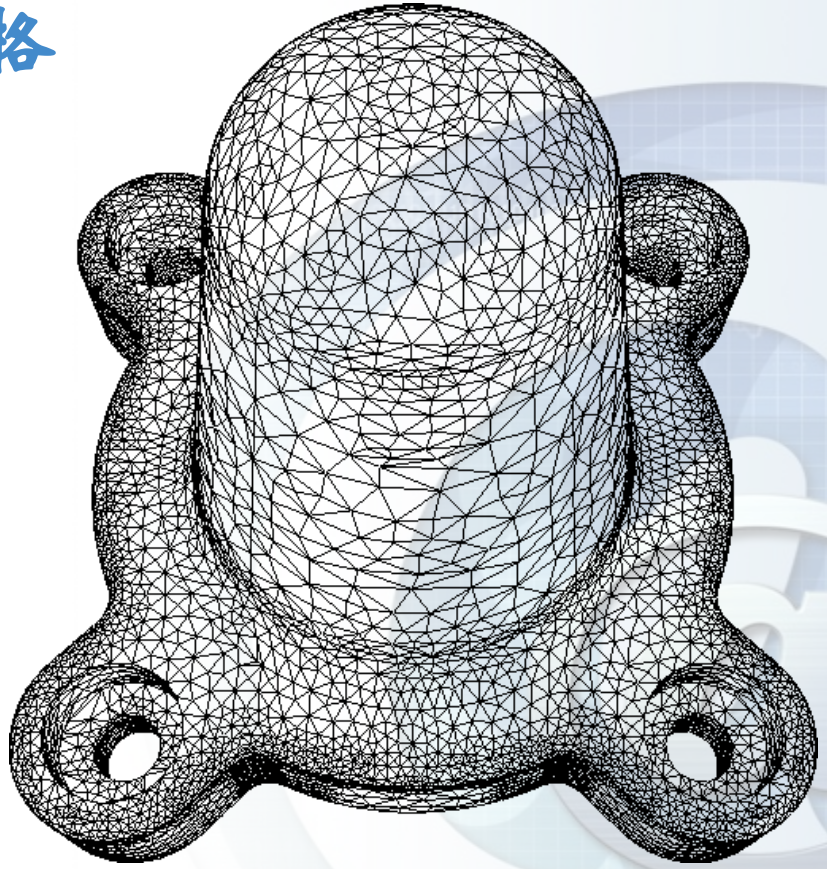
- 每个任务的通讯模式是相同的；
- 下面是否存在一个相同通讯模式？





非结构化通讯

- 没有一个统一的通讯模式
- 例如：无结构化网格



通讯判据

- 所有任务是否执行大致相当的通讯?
- 是否尽可能的局部通讯?
- 通讯操作是否能并行执行?
- 同步任务的计算能否并行执行?

3.组合

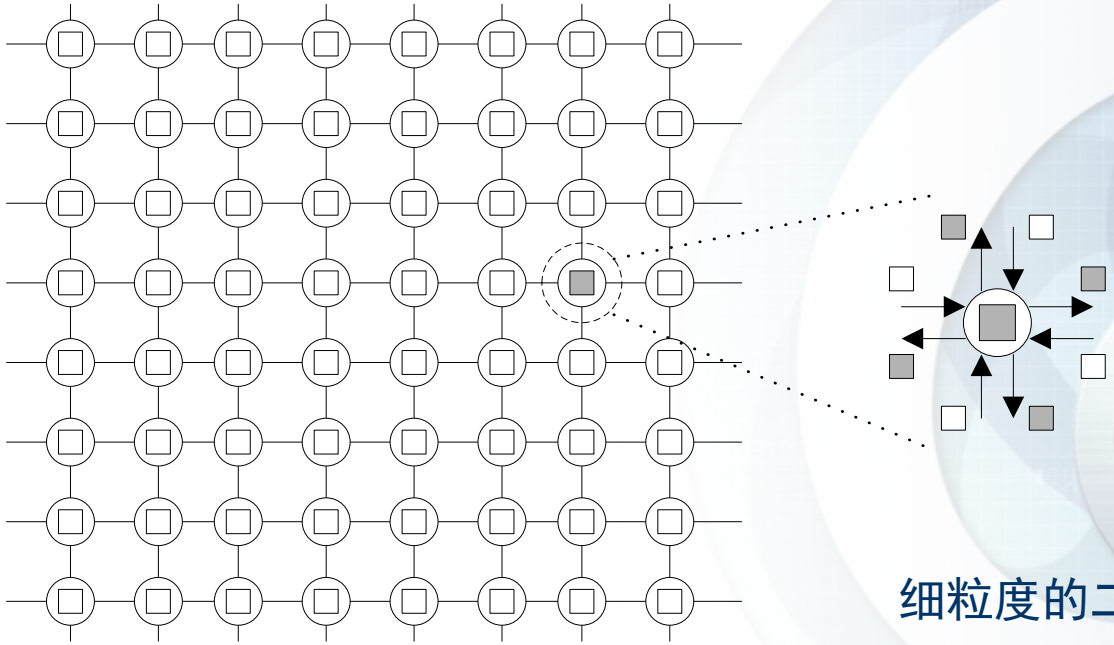
1.1 表面-容积效应

1.2 重复计算

1.3 组合判据

表面-容积效应

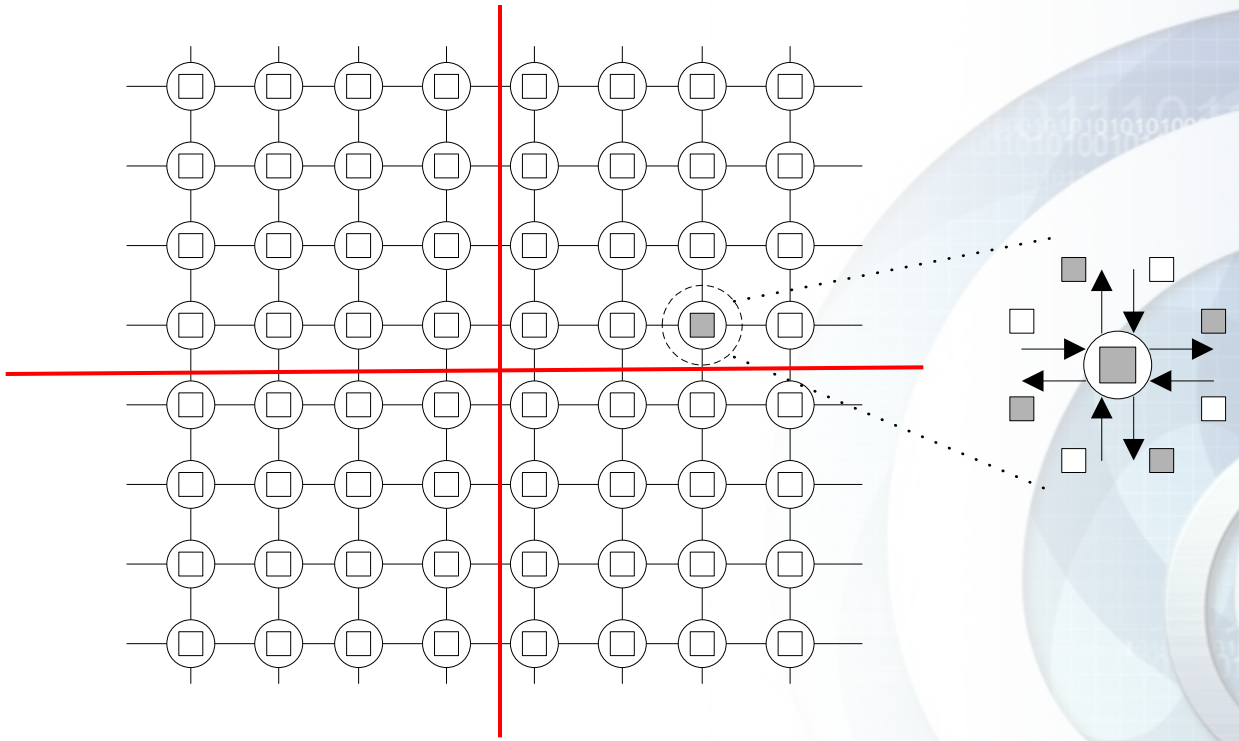
- 通讯量与任务子集的表面成正比，计算量与任务子集的体积成正比；
- 粗粒度划分的通信次数和通信量均比细粒度划分时有所下降；



细粒度的二维网格

表面-容积效应

❖粗粒度划分的通信次数和通信量均比细粒度划分时有所下降；



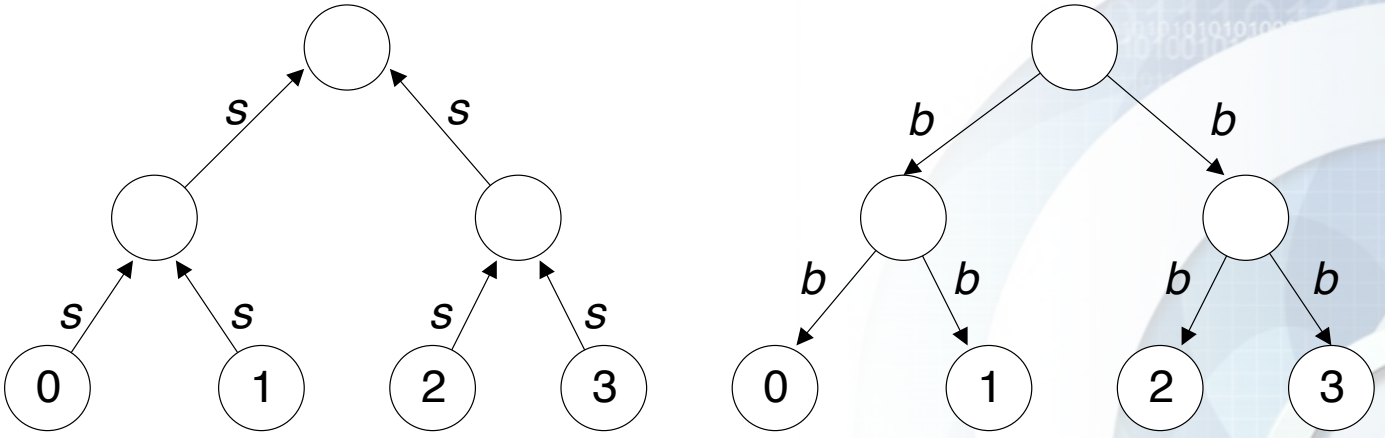
粗粒度的二维网格

重复计算

- 重复计算减少通讯量，但增加了计算量，应保持恰当的平衡；
- 重复计算的目标应减少算法的总运算时间；

重复计算

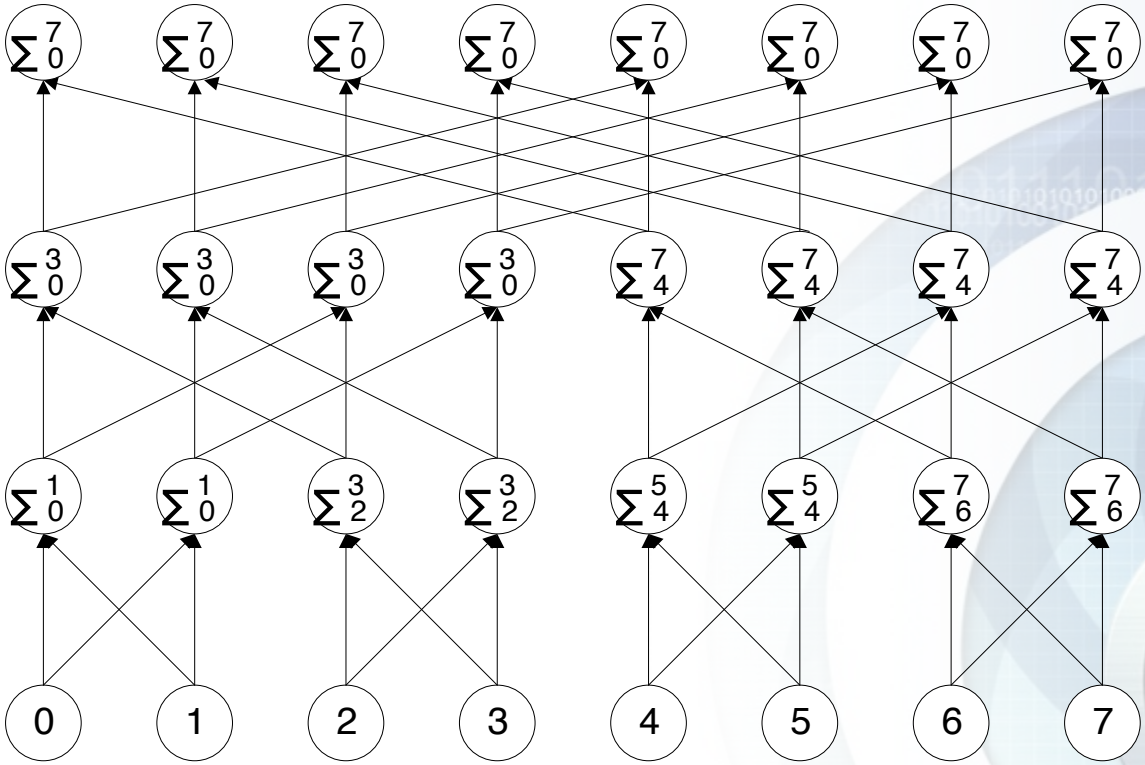
❖示例：二叉树上N个处理器求N个数的全和，要求每个处理器均保持全和。



二叉树上求和，共需2logN步

重复计算

- 示例：二叉树上N个处理器求N个数的全和，要求每个处理器均保持全和。



蝶式结构求和，使用了重复计算，共需logN步

组合判据

- 增加粒度是否减少了通讯成本?
- 重复计算是否已权衡了其得益?
- 是否保持了灵活性和可扩放性?
- 组合的任务数是否与问题尺寸成比例?
- 是否保持了类似的计算和通讯?
- 有没有减少并行执行的机会?

映射

4. 映射

4.1方法描述

4.2负载平衡算法

4.3任务调度算法

4.4映射判据

映射——方法描述

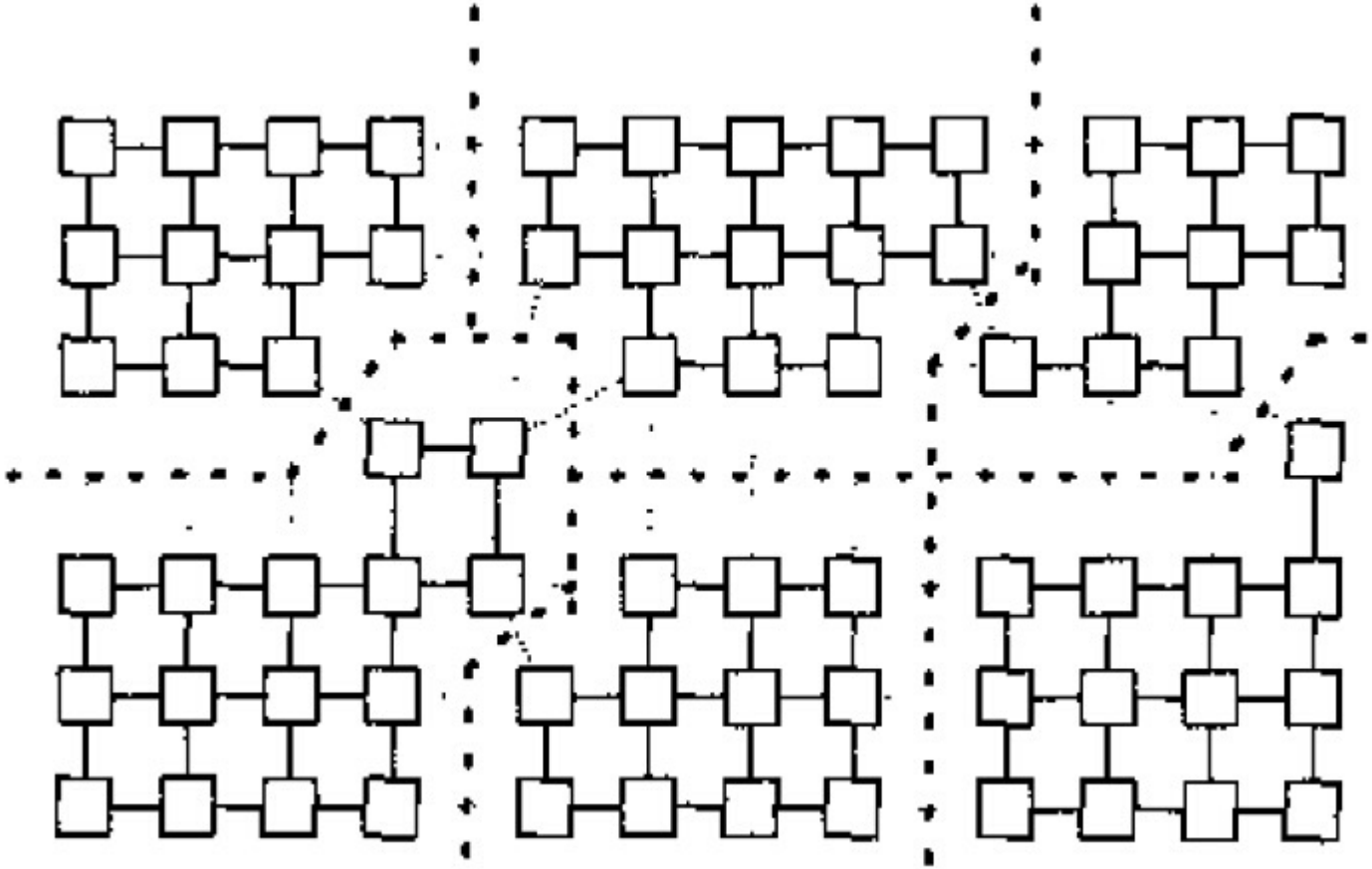
- 每个任务要映射到具体的处理器，定位到运行机器上；
- 任务数大于处理器数时，存在负载平衡和任务调度问题；
- 映射的目标：减少算法的执行时间
  - 并发的任务 → 不同的处理器
  - 任务之间存在高通讯的 → 同一处理器
- 映射实际是一种权衡，属于NP完全问题；

负载均衡算法

- 静态的：事先确定；
- 概率的：随机确定；
- 动态的：执行期间动态负载；
- 基于域分解的：
  - 递归对剖
  - 局部算法
  - 概率方法
  - 循环映射

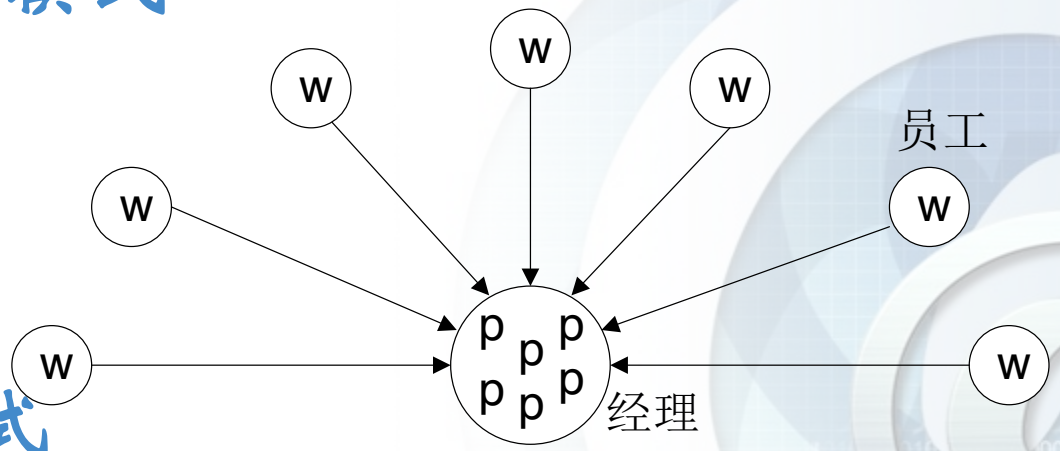
局部算法

❖网格问题中的负载平衡



任务调度算法

- 任务放在集中的或分散的任务池中，使用任务调度算法将池中的任务分配给特定的处理器。下面是两种常用调度模式：
- 经理/雇员模式



- 非集中模式

- 非集中模式也就是无中心管理者的分布式调度法。在每个处理器均维持一个任务池，这些任务池实际上就变成了可供请求者异步访问的分布的数据结构。

- 结束检测

  何时操作结束？

映射判据

- 采用集中式负载平衡方案，是否存在通讯瓶颈？
- 采用动态负载平衡方案，调度策略的成本如何？

小结

- 划分
  - 域分解和功能分解
- 通讯
  - 任务间的数据交换
- 组合
  - 任务的合并使得算法更有效
- 映射
  - 将任务分配到处理器，并保持负载平衡